



ELECTRICAL & ELECTRONICS ENGINEERING
EEE445 INTRODUCTION TO INTELLIGENT
CONTROL SYSTEMS

FINAL PROJECT REPORT
Design and Voltage Control of Buck Converter using
PID and Fuzzy PID Controller

200403059 - HALİL UZUNOĞLU

SUPERVISORS
Prof. Dr. Savaş ŞAHİN
R.A. Uğur SOYDEMİR

SPRING 2025

1. Abstract

This study presents the design, implementation, and performance comparison of classical PID and fuzzy logic-enhanced PID (Fuzzy PID) controllers for voltage regulation of a Buck Converter. The aim is to investigate how adaptive control methods improve system response under dynamic conditions. Initially, both control strategies were modelled in MATLAB/Simulink environment and tested through simulations. Then, the control algorithms were implemented on the STM32F429ZI microcontroller, and real-time tests were conducted using a custom hardware setup. A UART-based interface was developed to allow communication between the microcontroller and a graphical interface on the host computer. Key performance indicators such as rise time, settling time, maximum overshoot, and steady-state error were measured using step response experiments. Results demonstrate that the fuzzy PID controller significantly reduces rise time and overshoot while maintaining similar steady-state accuracy compared to the classical PID controller. This suggests that fuzzy logic integration offers enhanced adaptability and robustness for embedded power control systems.

2. Introduction

Voltage regulation plays a critical role in the performance and efficiency of power electronic systems, especially in applications where size, cost, and stability constraints are prominent. Among these systems, the Buck Converter is widely preferred for stepping down a high DC input voltage to a desired lower level with high efficiency. However, to achieve reliable and precise output, the control strategy applied to regulate the converter becomes a vital component of the overall design.[1]

Proportional–Integral–Derivative (PID) controllers are one of the most commonly used control techniques due to their simplicity, ease of implementation, and well-established theoretical background. Despite their widespread usage, classical PID controllers often face challenges in systems with nonlinear dynamics, parameter uncertainties, and time-varying loads—common characteristics in real-world Buck Converter applications.[2]

To overcome these limitations, intelligent control methods such as fuzzy logic have been introduced into control design. Fuzzy PID controllers extend traditional PID structures by dynamically adjusting the PID gains (K_p , K_i , K_d) using a rule-based fuzzy inference system. This allows the controller to respond adaptively to variations in the system's behavior, thereby improving transient and steady-state performance.

In this project, both classical PID and fuzzy PID controllers are designed, implemented, and evaluated on a Buck Converter. The design process includes MATLAB-based modeling and simulation, followed by real-time deployment on an STM32F429ZI microcontroller.[3] The effectiveness of both controllers is assessed through metrics such as rise time, overshoot, settling time, and steady-state error, highlighting the practical advantages of incorporating fuzzy logic in embedded control applications.

3. Materials and Methods

3.1. Hardware Requirements

3.1.1. Buck Converter

A Buck Converter is a widely used DC-DC step-down power converter designed to reduce the input voltage to a lower, regulated output level. Its core operation relies on high-frequency switching using components such as a MOSFET (or transistor), a flyback diode, an inductor, and an output capacitor. The MOSFET operates in a rapid on-off cycle controlled by a PWM signal. When the switch is closed, energy is stored in the inductor's magnetic field; when opened, the energy is released to the load through the diode, smoothing current flow.

The voltage output V_{out} of a Buck Converter is primarily determined by the duty cycle D of the PWM:

$$V_{out} = D \times V_{in} \quad (1)$$

In this study, the Buck Converter was custom-built and tuned for operation under controlled laboratory conditions. The system was designed to respond quickly to changes in control input, making it ideal for testing both classical and fuzzy PID algorithms. The output voltage is measured via an analog voltage divider and fed into the microcontroller's ADC (Analog-to-Digital Converter) for feedback control.[4]

Parameters of the converter:

- Input Voltage: 15V_{DC}
- Output Range: 1V to 10V (depending on duty cycle)
- Switching Frequency: ~100 kHz
- Load: Fixed resistive load (with optional step-change)
- Output current maximum: 0.5A
- Inductor current ripple: 0.15A
- Capacitor voltage ripple: 0.05V

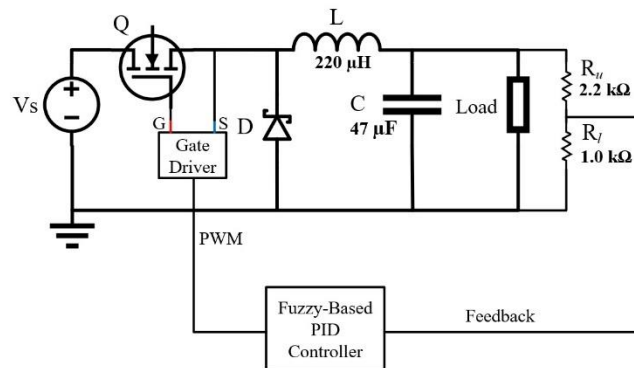


Figure 1. Buck Converter Schematic

The inductor calculation is given in Equation 2.

$$L = \frac{(V_{in} - V_{out}) \cdot D}{f_{sw} \cdot \Delta I_L} = \frac{(10 - 5) \cdot 0.5}{(100 \times 10^3) \cdot 0.15} = 166.7 \mu H \quad (2)$$

The process to limit the capacitor output ripple is given in Equation 3.

$$C_{out} \geq \frac{\Delta I_L}{8 \cdot f_{sw} \cdot \Delta V_{out}} = \frac{0.15}{8 \cdot (100 \times 10^3) \cdot 0.05} = 3.75 \mu F \quad (3)$$

Buck converter steady state equations are found below.

$$dt diL = 1L Vin \cdot u - vC \quad (4)$$

$$dvC dt = 1C (iL - vC Rload) \quad (5)$$

$$x' = Ax + BD, \quad (6)$$

$$y = Cx, \quad (7)$$

$$x = \begin{bmatrix} iL \\ vC \end{bmatrix}, \quad (8)$$

$$A = \begin{bmatrix} 0 & -1/L1 \\ 1/C & -1/(RloadC) \end{bmatrix}, \quad (9)$$

$$B = \begin{bmatrix} Vin/L \\ 0 \end{bmatrix}, \quad (10)$$

$$C = [0 \quad 1]. \quad (11)$$

3.1.2. Microcontroller

The STM32F429ZI is a high-performance 32-bit ARM Cortex-M4 microcontroller developed by STMicroelectronics. It features a floating-point unit (FPU), large memory capacity, and high-speed peripherals, making it particularly well-suited for real-time control applications such as power electronics regulation.

In this project, the STM32F429ZI performs three major tasks:

- **Control Loop Execution:** Executes either the classical PID algorithm or the fuzzy PID algorithm in real-time, computing the appropriate PWM duty cycle.
- **ADC Sampling:** Reads the Buck Converter output voltage using its 12-bit ADC for feedback in the control loop.
- **UART Communication:** Interfaces with a PC-based MATLAB GUI for monitoring, parameter tuning, and real-time data visualization.

The PWM signal used to control the converter's switching transistor is generated via TIM1 PWM output, which is configured for high-resolution pulse-width modulation. ADC reads are filtered using a moving average technique to reduce measurement noise. UART communication (921600 baud rate) ensures fast data transfer for effective interaction between the microcontroller and MATLAB.

Key features utilized:

- TIM1 for PWM generation
- ADC1 for analog voltage sampling

- USART1 for serial communication

The microcontroller firmware was developed using STM32CubeIDE and HAL libraries in C, and includes modular code for easy switching between PID and fuzzy PID control modes.



Figure 2. STM32F429ZI Discovery Board

3.2. Control Strategies

3.2.1. Classical PID Control Approach

Classical PID (Proportional–Integral–Derivative) control is a time-tested method widely applied in industrial and embedded systems due to its simple structure and effective regulation capability. The main objective of a PID controller is to minimize the error between a desired setpoint and the actual output by calculating a corrective control signal based on three components: the proportional term responds to the current error, the integral term addresses the accumulated past errors to eliminate steady-state offset, and the derivative term predicts the future trend by observing the rate of error change. These components are linearly combined through fixed gain coefficients K_p , K_i and K_d respectively, to compute the final control output.

In this project, a PID controller was implemented on the STM32F429ZI microcontroller to regulate the output voltage of a Buck Converter. The controller continuously reads the output voltage via ADC, calculates the error with respect to the reference voltage, and applies the PID equation to determine a new PWM duty cycle. The updated duty cycle modulates the switching of the converter to maintain the output within desired limits. Although this method performs reasonably well in stable operating conditions, its effectiveness can deteriorate when the system is subject to parameter variations or sudden load changes, due to the fixed nature of the controller gains.

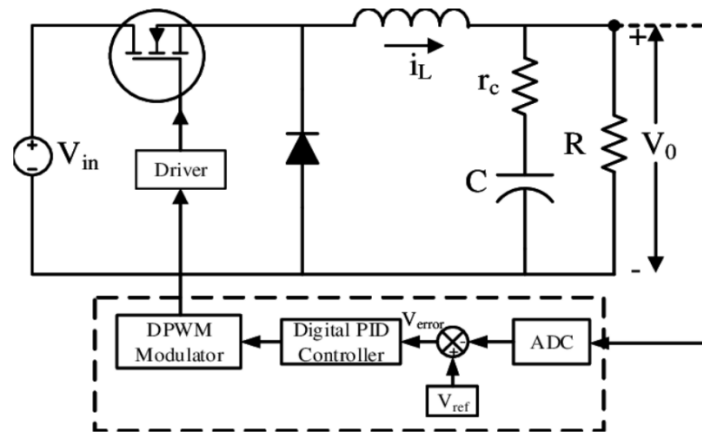


Figure 3. PID Control Schematic

3.2.2. Fundamentals of Fuzzy Logic Control

Fuzzy logic control presents a rule-based approach to system regulation, particularly useful in applications where the mathematical model of the system is incomplete, highly nonlinear, or affected by uncertainties. Unlike classical controllers that rely on precise numeric relationships, fuzzy logic uses linguistic variables and human-like reasoning to determine the control action. This is achieved through a structured process involving fuzzification, inference, and defuzzification.

In the fuzzification step, crisp numerical inputs such as error and error rate are transformed into fuzzy variables using predefined membership functions. These inputs are then evaluated within a rule base composed of "IF-THEN" statements, which mimic expert control behaviour. For instance, a typical rule might state: "If error is negative small and error rate is zero, then apply medium positive correction." The output of this inference stage is a fuzzy set, which is converted back into a precise numeric control value through defuzzification, typically using the centroid method.

In this project, a fuzzy logic controller was developed using MATLAB's Fuzzy Logic Toolbox. The controller utilizes two inputs: the instantaneous error between the reference and output voltages, and the rate of change of this error. By incorporating fuzzy rules, the controller can make decisions that are more tolerant to uncertainties and better suited to nonlinear behaviours compared to rigid classical approaches.

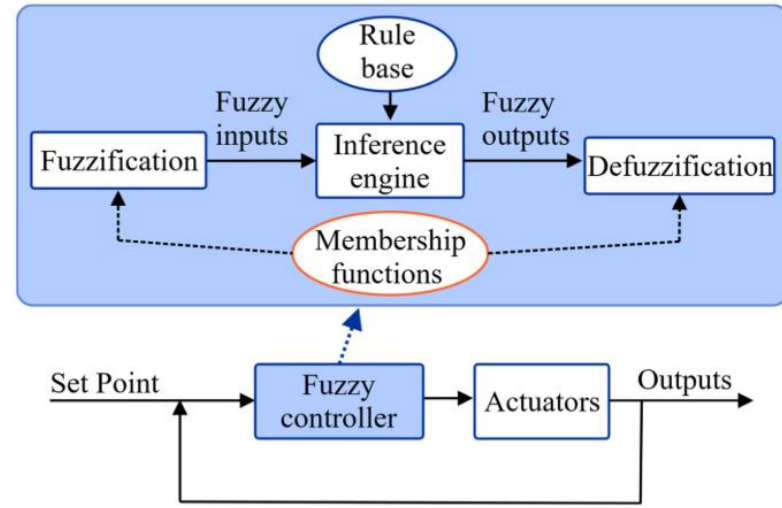


Figure 4. Fuzzy Logic Control Schematic

3.2.3 Fuzzy PID Control

The fuzzy PID control strategy combines the structural benefits of the PID controller with the adaptability of fuzzy logic. Rather than using static gain values, the fuzzy PID dynamically adjusts the coefficients K_p , K_i and K_d in real time based on system conditions. This allows the controller to respond effectively to a wide range of disturbances and variations in operating points.

In the proposed design, two input variables error and error rate are fed into a fuzzy inference system that outputs three fuzzy-tuned parameters: K_p , K_i and K_d . The inference mechanism is based on a set of heuristic rules that relate the current system behaviour to appropriate gain adjustments. For example, when the error is large and increasing, a high proportional gain might be selected to drive the system rapidly toward the setpoint. Conversely, when the error is small but persistent, the integral gain is increased to eliminate steady-state error. If the system is oscillating or responding too abruptly, the derivative gain is adjusted to dampen the response.[5]

The fuzzy inference system was designed and simulated in MATLAB and then integrated with the embedded C code running on the STM32 microcontroller. During operation, the system continuously computes the error and its derivative, determines the optimal PID gains through the fuzzy rule base, and applies the updated gains in the PID formula. This adaptive approach significantly enhances the controller's flexibility and improves the stability and speed of the voltage regulation, particularly under varying load conditions.

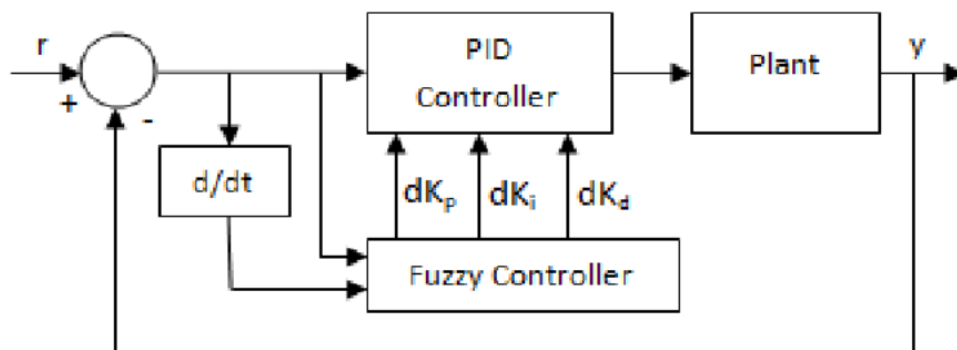


Figure 5. Fuzzy PID Control Schematic[6]

3.3. System Design and Implementation

3.3.1. Design of the Fuzzy PID Controller Inference System in MATLAB

The fuzzy PID controller was initially designed and tested in the MATLAB environment using the Fuzzy Logic Toolbox. The controller is structured to compute the proportional (K_p), integral (K_i), and derivative (K_d) gains dynamically, based on real-time measurements of the system's behaviour. Two input variables were defined for the fuzzy inference system: Error (the difference between the reference voltage and the measured output voltage), and Error Rate (the rate of change of the error over time). These two variables together provide the controller with sufficient insight into both the magnitude and trend of the deviation.

To reduce complexity while maintaining control performance, both inputs were modelled using five membership functions each. These linguistic categories—Negative Big (NB), Negative Small (NS), Zero (Z), Positive Small (PS), and Positive Big (PB)—were shaped using triangular and trapezoidal functions to provide both smoothness and sensitivity across the input range. Each output parameter (K_p , K_i and K_d) is also defined using five membership functions, allowing for flexible tuning across a wide range of operating conditions.

For each output parameter, a 5×5 fuzzy rule base was constructed, yielding 25 rules per output and a total of 75 rules across the full fuzzy PID controller. These rule tables were created heuristically to reflect expert reasoning. For instance, if the error is large and increasing, K_p is reduced to prevent overshoot, while K_d is increased to improve damping. Conversely, when the error is small but persistent, K_i is emphasized to eliminate steady-state error.

The inference system uses a Mamdani-type inference mechanism, which is intuitive and interpretable, and the output is defuzzified using the centroid (center of gravity) method. This generates crisp values for each PID coefficient at every control cycle. These coefficients are then directly applied in the PID control formula to compute the control signal. The fuzzy inference system developed in MATLAB thus provides an adaptive and intelligent layer on top of the classical PID control structure, significantly improving the system's ability to handle nonlinearities and uncertainties.

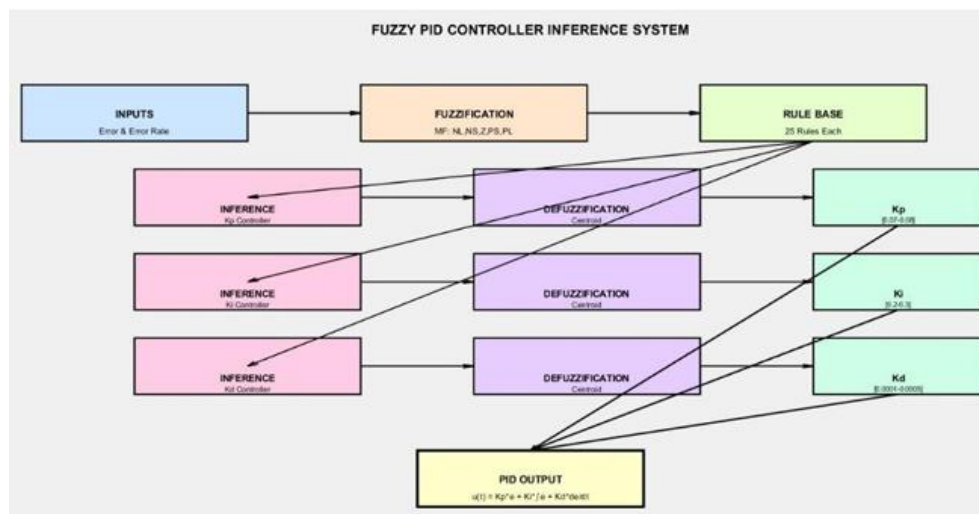


Figure 6. Design of Fuzzy PID Controller Inference System

3.3.2. Hardware Integration with STM32F429ZI

After the fuzzy inference system was modelled in MATLAB, it was integrated into a real-time embedded system using the STM32F429ZI microcontroller. The controller executes the main control algorithm, reads the output voltage via ADC, and adjusts the PWM duty cycle driving the Buck Converter. The conversion between the fuzzy rules and embedded C code was done manually, replicating the behaviour of the rule base and membership functions through conditional logic and lookup tables.

The STM32F429ZI was chosen due to its high processing power and peripherals such as 12-bit ADC, high-speed timers, and UART. The microcontroller reads voltage feedback, calculates the error and error rate, determines the appropriate PID gains through fuzzy logic, and calculates the control output accordingly. The PWM signal is generated using.

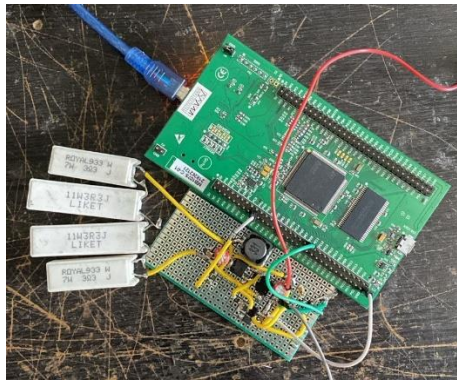


Figure 7. Real System of the Buck Converter

3.3.3. Fuzzy Membership Functions and Rule Tables

In the developed fuzzy PID system, both input variables Error and Error Rate are defined using five membership functions: Negative Big (NB), Negative Small (NS), Zero (Z), Positive Small (PS), and Positive Big (PB). These linguistic values are modelled with triangular and trapezoidal membership functions designed to capture both small and large deviations in system behaviour.

For the output parameters K_p , K_i and K_d , separate fuzzy rule bases were constructed. Each rule base is defined as a 5×5 matrix, resulting in 25 rules per output parameter. In total, the fuzzy system utilizes 75 rules. This structure allows each gain to be tuned independently based on a compact yet effective decision space.

The fuzzy inference engine uses Mamdani-type inference and centroid defuzzification to generate crisp values for each gain. Figures 13, 14, and 15 in the report illustrate how each parameter's output varies based on different combinations of error and error rate via MATLAB's Rule Viewer. This design allows the controller to dynamically adapt its behaviour in real time and respond more effectively under varying operating conditions.

3.3.4. Real-Time Control Application

In the embedded system, the fuzzy PID control runs in a loop triggered periodically. The process begins with ADC sampling of the converter's output voltage, followed by error and error rate computation. These values are processed through the fuzzy logic engine to derive the

appropriate gains. The resulting control signal is converted into a PWM signal using TIM1 and applied to the Buck Converter's switching device.

Additionally, a UART-based interface allows real-time monitoring and control from a MATLAB GUI. The interface supports commands such as "DUTY: xx" to manually set PWM duty or "ADC?" to retrieve the current filtered voltage reading. These UART features aid both debugging and visualization during experimental testing.

This complete system starting from MATLAB modelling to embedded execution successfully demonstrates how fuzzy logic can be integrated into a microcontroller-based control system for improved voltage regulation in power electronics.

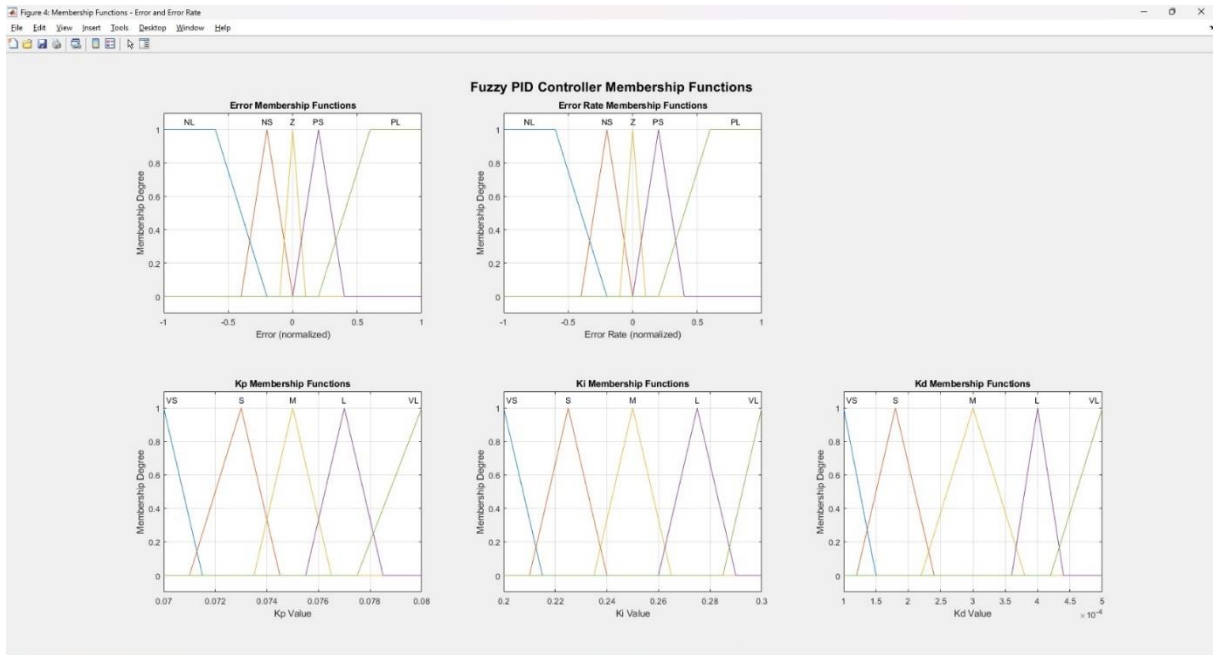


Figure 8. Membership Functions of the Error, Error Rate, K_p , K_i and K_d

3.3.3.1. Linguistic Variables and Fuzzy PID Controller Rule Tables

One of the fundamental components of the fuzzy PID controller design is the definition of linguistic variables and the construction of fuzzy rule tables. These elements form the core of the decision-making process in a fuzzy inference system. In this study, two input variables were used: Error and Error Rate. For each of these inputs, five linguistic terms were defined: Negative Big (NB), Negative Small (NS), Zero (Z), Positive Small (PS), and Positive Big (PB). These terms allow the system to interpret a wide range of control scenarios using approximate reasoning, rather than precise numerical values.

Similarly, the three output variables K_p , K_i and K_d are also defined with the same five linguistic terms. These outputs represent the proportional, integral, and derivative gains that are dynamically adjusted by the controller at each control cycle. By assigning fuzzy sets to each of these variables, the controller is able to respond flexibly and intelligently to changing system dynamics.

For each output parameter, a separate 5×5 rule table was developed. These tables define the control strategy by associating every possible combination of Error and Error Rate with a corresponding output gain level. For instance:

- When the Error is NB (large negative) and the Error Rate is also NB (rapidly decreasing), the controller may assign a relatively large K_p value to quickly bring the system back to setpoint.
- Conversely, if both Error and Error Rate are PB (large and increasing), a smaller K_p may be selected to avoid overshooting.
- The K_i rules are more active when Error is small but persistent.
- The K_d rules are triggered in scenarios involving rapid fluctuations, to improve damping and suppress oscillations.

Each rule table consists of 25 rules, resulting in a total of 75 fuzzy rules across all three PID coefficients. These rules were designed based on heuristic control knowledge and tested through simulations in MATLAB. Figures in the report (e.g., Figure 10, Figure 11, and Figure 12) illustrate the structure of the rule tables for K_p , K_i and K_d , respectively.

This linguistic structure and rule base enable the fuzzy PID controller to operate with a high degree of adaptability and robustness. Unlike classical PID controllers with fixed gain values, the fuzzy approach continually adjusts its behaviour according to the system's real-time state, resulting in smoother transitions, reduced overshoot, and better steady-state accuracy.



Figure 9. Linguistic Variables and Fuzzy PID Controller Rule Tables

3.3.3.2. Rules

The fuzzy PID controller operates based on rule tables that translate qualitative assessments of the system's state into precise control actions. Each of the three PID gain parameters proportional (K_p), integral (K_i), and derivative (K_d) is controlled through a separate set of fuzzy rules. These rules are defined using combinations of the input linguistic variables Error and Error Rate, each of which has five fuzzy terms: Negative Big (NB), Negative Small (NS), Zero (Z), Positive Small (PS), and Positive Big (PB).

Rules for K_p :

The proportional gain K_p is primarily responsible for reacting to the instantaneous error. The fuzzy rules for K_p are designed to produce a strong control effort when the system is far from the reference and reduce the gain when the system is approaching stability to avoid overshoot. For example:

- If Error is NB and Error Rate is NB, then K_p is PB $\rightarrow$ system is far from setpoint and diverging fast, aggressive correction is needed.
- If Error is Z, regardless of Error Rate, K_p is Z or NS $\rightarrow$ minimal or no correction needed.
- If Error is PB and Error Rate is PB, then K_p is NB $\rightarrow$ system is overshooting, so gain is reduced to prevent instability.

The full 5 $\times$ 5 rule table for K_p is shown in Figure 10.

```
rules = [
    1 1 4 1 1; % If Error is NL and ErrorRate is NL then Kp is L |
    1 2 4 1 1; % If Error is NL and ErrorRate is NS then Kp is L
    1 3 3 1 1; % If Error is NL and ErrorRate is Z then Kp is M
    1 4 2 1 1; % If Error is NL and ErrorRate is PS then Kp is S
    1 5 1 1 1; % If Error is NL and ErrorRate is PL then Kp is VS
    2 1 4 1 1; % If Error is NS and ErrorRate is NL then Kp is L
    2 2 3 1 1; % If Error is NS and ErrorRate is NS then Kp is M
    2 3 3 1 1; % If Error is NS and ErrorRate is Z then Kp is M
    2 4 2 1 1; % If Error is NS and ErrorRate is PS then Kp is S
    2 5 1 1 1; % If Error is NS and ErrorRate is PL then Kp is VS
    3 1 3 1 1; % If Error is Z and ErrorRate is NL then Kp is M
    3 2 3 1 1; % If Error is Z and ErrorRate is NS then Kp is M
    3 3 3 1 1; % If Error is Z and ErrorRate is Z then Kp is M
    3 4 3 1 1; % If Error is Z and ErrorRate is PS then Kp is M
    3 5 3 1 1; % If Error is Z and ErrorRate is PL then Kp is M
    4 1 2 1 1; % If Error is PS and ErrorRate is NL then Kp is S
    4 2 2 1 1; % If Error is PS and ErrorRate is NS then Kp is S
    4 3 3 1 1; % If Error is PS and ErrorRate is Z then Kp is M
    4 4 3 1 1; % If Error is PS and ErrorRate is PS then Kp is M
    4 5 4 1 1; % If Error is PS and ErrorRate is PL then Kp is L
    5 1 1 1 1; % If Error is PL and ErrorRate is NL then Kp is VS
    5 2 2 1 1; % If Error is PL and ErrorRate is NS then Kp is S
    5 3 3 1 1; % If Error is PL and ErrorRate is Z then Kp is M
    5 4 4 1 1; % If Error is PL and ErrorRate is PS then Kp is L
    5 5 4 1 1; % If Error is PL and ErrorRate is PL then Kp is L
];
```

Figure 10. Rules for K_p

Rules for K_i :

The integral gain K_i addresses the accumulation of small persistent errors over time. Its rules are crafted to act when the error is small but stable, which indicates a potential steady-state offset. Some representative rules include:

- If Error is Z and Error Rate is Z, then K_i is PB $\rightarrow$ correction for a possible steady-state error.
- If Error is NS and Error Rate is PS, then K_i is PM $\rightarrow$ slow response toward setpoint.
- If Error is PB and Error Rate is PB, then K_i is NB $\rightarrow$ to prevent wind-up and oscillation during overshoot.

These rules prevent the integral term from overcompensating in fast-changing conditions while still correcting for long-term drift. See Figure 11 for the full rule base.

```

rules = [
    1 1 3 1 1; % If Error is NL and ErrorRate is NL then Ki is M
    1 2 4 1 1; % If Error is NL and ErrorRate is NS then Ki is L
    1 3 5 1 1; % If Error is NL and ErrorRate is Z then Ki is VL
    1 4 4 1 1; % If Error is NL and ErrorRate is PS then Ki is L
    1 5 3 1 1; % If Error is NL and ErrorRate is PL then Ki is M
    2 1 2 1 1; % If Error is NS and ErrorRate is NL then Ki is S
    2 2 3 1 1; % If Error is NS and ErrorRate is NS then Ki is M
    2 3 4 1 1; % If Error is NS and ErrorRate is Z then Ki is L
    2 4 3 1 1; % If Error is NS and ErrorRate is PS then Ki is M
    2 5 2 1 1; % If Error is NS and ErrorRate is PL then Ki is S
    3 1 2 1 1; % If Error is Z and ErrorRate is NL then Ki is S
    3 2 2 1 1; % If Error is Z and ErrorRate is NS then Ki is S
    3 3 3 1 1; % If Error is Z and ErrorRate is Z then Ki is M
    3 4 2 1 1; % If Error is Z and ErrorRate is PS then Ki is S
    3 5 2 1 1; % If Error is Z and ErrorRate is PL then Ki is S
    4 1 2 1 1; % If Error is PS and ErrorRate is NL then Ki is S
    4 2 3 1 1; % If Error is PS and ErrorRate is NS then Ki is M
    4 3 4 1 1; % If Error is PS and ErrorRate is Z then Ki is L
    4 4 3 1 1; % If Error is PS and ErrorRate is PS then Ki is M
    4 5 2 1 1; % If Error is PS and ErrorRate is PL then Ki is S
    5 1 3 1 1; % If Error is PL and ErrorRate is NL then Ki is M
    5 2 4 1 1; % If Error is PL and ErrorRate is NS then Ki is L
    5 3 5 1 1; % If Error is PL and ErrorRate is Z then Ki is VL
    5 4 4 1 1; % If Error is PL and ErrorRate is PS then Ki is L
    5 5 3 1 1; % If Error is PL and ErrorRate is PL then Ki is M
];

```

Figure 11. Rules for K_i

Rules for K_d :

The derivative gain K_d contributes to the system's damping by anticipating future behavior based on the rate of change in the error. Its rules are particularly important when the system exhibits oscillatory tendencies or sudden error changes. Some examples include:

- If Error is NB and Error Rate is PB, then K_d is PB $\rightarrow$ large error is reducing quickly; derivative term adds stabilization.
- If Error is Z and Error Rate is NB, then K_d is PS $\rightarrow$ small deviation with increasing error: moderate damping.
- If Error and Error Rate are both Z, then K_d is Z $\rightarrow$ system is near steady-state, no derivative action needed.

These rules ensure that derivative action is only applied when beneficial and do not amplify noise unnecessarily. The complete rule set is given in Figure 12.

```

rules = [
    1 1 4 1 1; % If Error is NL and ErrorRate is NL then Kd is L
    1 2 3 1 1; % If Error is NL and ErrorRate is NS then Kd is M
    1 3 2 1 1; % If Error is NL and ErrorRate is Z then Kd is S
    1 4 1 1 1; % If Error is NL and ErrorRate is PS then Kd is VS
    1 5 1 1 1; % If Error is NL and ErrorRate is PL then Kd is VS
    2 1 3 1 1; % If Error is NS and ErrorRate is NL then Kd is M
    2 2 3 1 1; % If Error is NS and ErrorRate is NS then Kd is M
    2 3 2 1 1; % If Error is NS and ErrorRate is Z then Kd is S
    2 4 1 1 1; % If Error is NS and ErrorRate is PS then Kd is VS
    2 5 1 1 1; % If Error is NS and ErrorRate is PL then Kd is VS
    3 1 2 1 1; % If Error is Z and ErrorRate is NL then Kd is S
    3 2 2 1 1; % If Error is Z and ErrorRate is NS then Kd is S
    3 3 3 1 1; % If Error is Z and ErrorRate is Z then Kd is M
    3 4 2 1 1; % If Error is Z and ErrorRate is PS then Kd is S
    3 5 2 1 1; % If Error is Z and ErrorRate is PL then Kd is S
    4 1 2 1 1; % If Error is PS and ErrorRate is NL then Kd is S
    4 2 2 1 1; % If Error is PS and ErrorRate is NS then Kd is S
    4 3 2 1 1; % If Error is PS and ErrorRate is Z then Kd is S
    4 4 3 1 1; % If Error is PS and ErrorRate is PS then Kd is M
    4 5 3 1 1; % If Error is PS and ErrorRate is PL then Kd is M
    5 1 1 1 1; % If Error is PL and ErrorRate is NL then Kd is VS
    5 2 1 1 1; % If Error is PL and ErrorRate is NS then Kd is VS
    5 3 2 1 1; % If Error is PL and ErrorRate is Z then Kd is S
    5 4 3 1 1; % If Error is PL and ErrorRate is PS then Kd is M
    5 5 4 1 1; % If Error is PL and ErrorRate is PL then Kd is L
];

```

Figure 12. Rules for K_d

3.3.3.3. Visualization of Rule Activation via Fuzzy Rule Viewer

To provide insight into how the fuzzy PID controller dynamically determines gain values based on real-time system states, the MATLAB Fuzzy Logic Toolbox offers a graphical Rule Viewer interface. This tool visually demonstrates which rules are being activated for a given set of input values and how the corresponding outputs are computed.

Figure 13 displays the rule viewer for the proportional gain K_p . When specific Error and Error Rate values are input into the system, the viewer highlights the intersecting rules in the fuzzy rule base. The activated input membership functions are shown as shaded areas, and the resulting fuzzy output for K_p is visualized as a weighted aggregate of rule contributions. The final crisp output (defuzzified value) is indicated by a vertical red line on the output axis.

This visualization clearly demonstrates how K_p increases in response to large, opposite-signed error trends (e.g., Error = NB, Error Rate = PB) to accelerate convergence, and decreases when the system approaches stability.



Figure 13. K_p in the Fuzzy PID Rule Viewer

In Figure 14, the integral gain K_i is shown in the Rule Viewer. The viewer reveals how K_i is primarily activated in cases where Error remains near Zero and Error Rate is small, which typically suggests the presence of a steady-state error. The graphical interface highlights how the membership functions overlap and contribute to the final K_i value.

This selective activation ensures that the integral action is applied only when needed preventing integral windup and maintaining stability.



Figure 14. K_i in the Fuzzy PID Rule Viewer

Figure 15 illustrates the behavior of the derivative gain K_d . The Rule Viewer highlights its sensitivity to rapid changes in error. When Error Rate is large (positive or negative), K_d

increases to provide damping and prevent oscillations. Conversely, in steady or slow-changing conditions, the output from K_d remains low, reducing unnecessary derivative action.

Through this interface, it becomes evident how the fuzzy controller adaptively modulates K_d in response to dynamic shifts, improving transient behaviour and ensuring smoother system responses.



Figure 15. K_d in the Fuzzy PID Rule Viewer

3.3.3.4. Membership Functions

Membership functions play a central role in the performance and sensitivity of a fuzzy inference system. They define how crisp numerical inputs are mapped to fuzzy linguistic values and how the resulting fuzzy output sets are translated into meaningful control actions. In this study, the membership functions were carefully designed for both the input variables (Error and Error Rate) and the output variables in order to achieve reliable and adaptive voltage regulation for the Buck Converter.

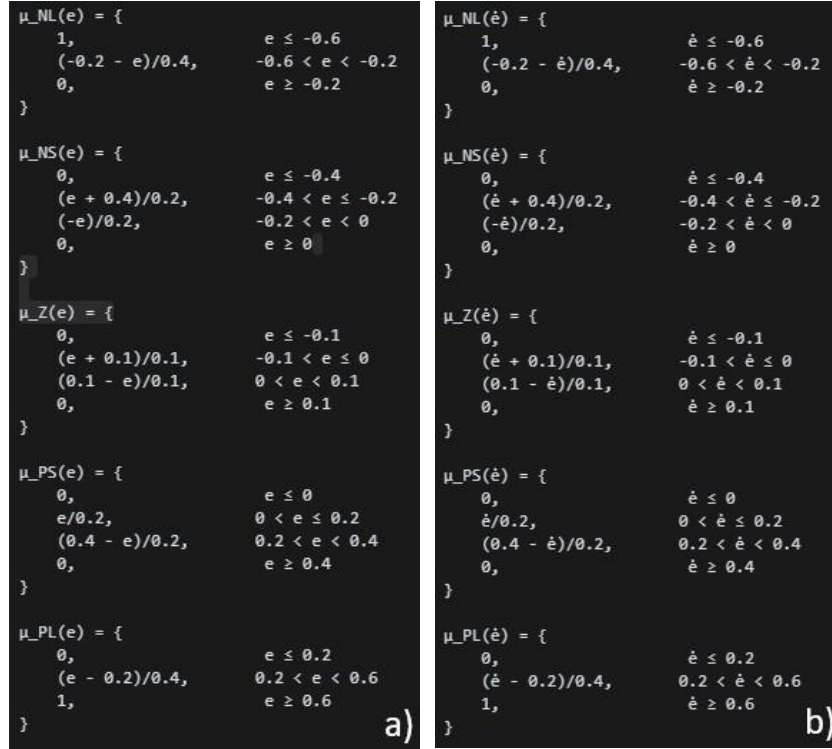


Figure 16. Membership Functions of Error (a) and Error Rate (b)

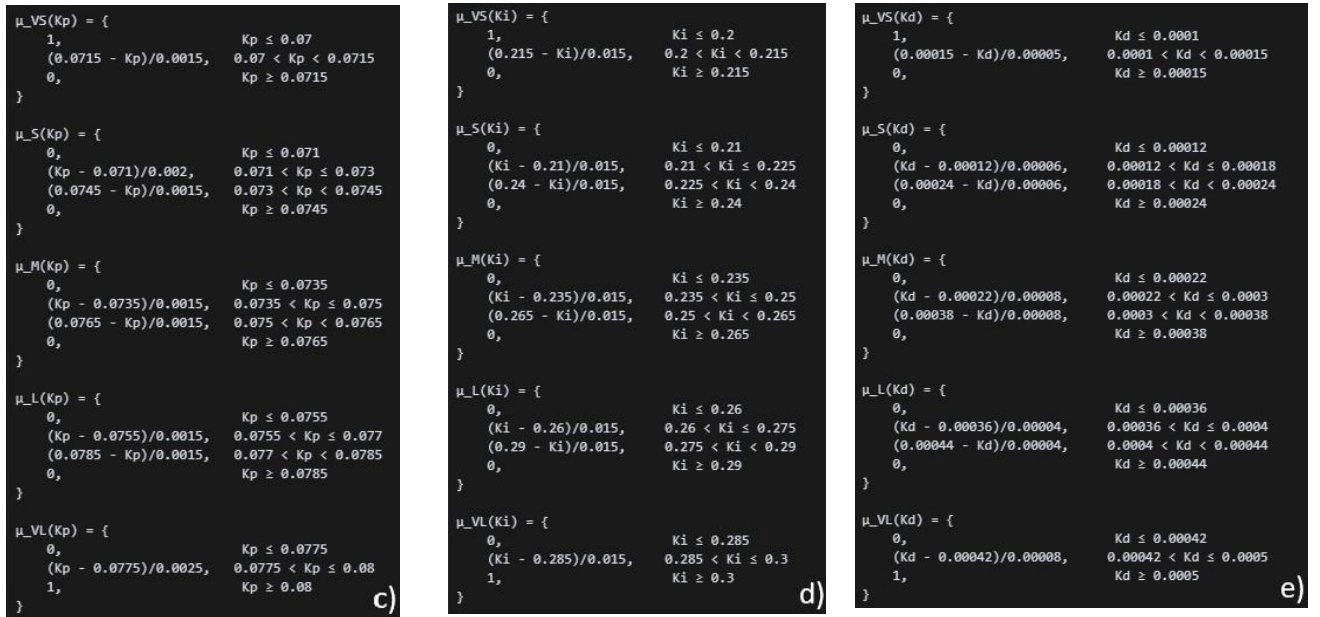


Figure 17. Membership Functions of K_p (c), K_i (d) and K_d (e)

The fuzzy PID controller designed in MATLAB was implemented in real-time on the STM32F429ZI microcontroller to control the output voltage of a Buck Converter. The controller calculates Error and Error Rate from ADC readings and uses these inputs in a simplified fuzzy inference engine coded in C. The resulting K_p , K_i and K_d values are applied in the PID equation to compute the PWM duty cycle driving the converter.

The control loop runs periodically, performing ADC sampling, fuzzy rule evaluation, and PWM update using the TIM1 timer. The duty cycle is limited to 90% to ensure safe operation. To monitor system behaviour, a UART interface is used to communicate with a MATLAB GUI, enabling real-time data logging and user control.

The full hardware setup is shown in Figure 18, demonstrating successful integration of fuzzy logic control into an embedded power system. This implementation allows adaptive voltage regulation in response to changing conditions, validating the effectiveness of fuzzy PID control in practical applications.

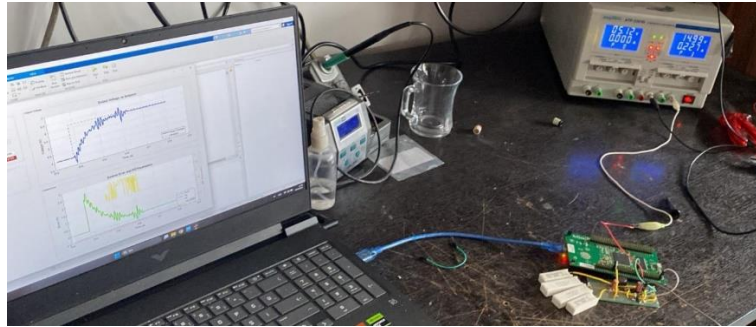


Figure 18. Setup of the Fuzzy PID Controller Implementation in Real-Time

4. Experimental Results

To validate the effectiveness of the implemented controllers, experimental tests were carried out using the Buck Converter setup under identical conditions for both control strategies. The system was subjected to step changes in the reference voltage, and the output response was observed via a MATLAB-based graphical interface communicating with the STM32F429ZI microcontroller through UART.

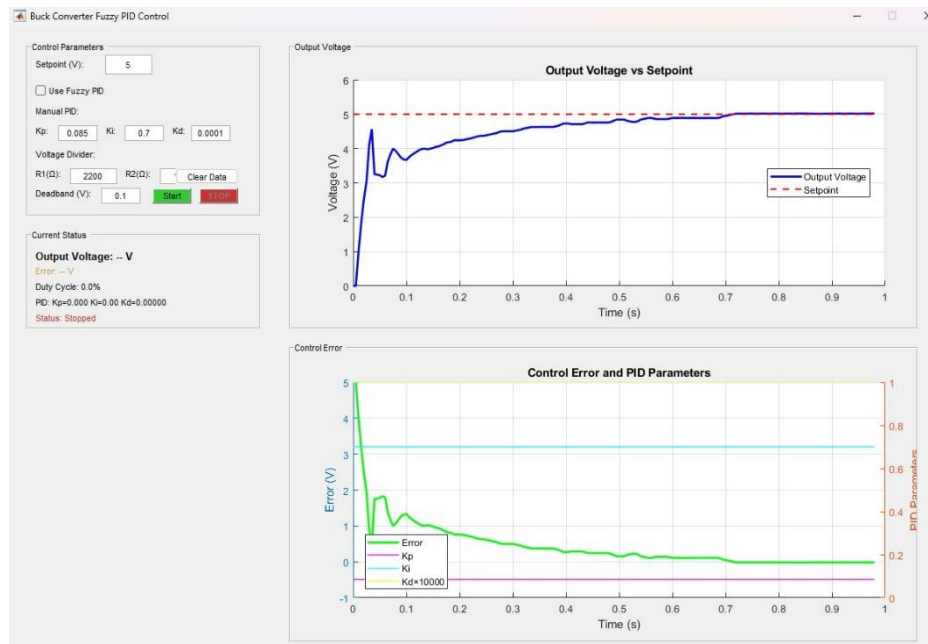


Figure 19. GUI of the Voltage Control of Buck Converter using PID Controller

4.1. Classical PID Controller Performance

The classical PID controller was tested first. Fixed gain values for K_p , K_i and K_d were manually tuned to balance speed and stability. As shown in Figure 19, a GUI was used to input reference values and visualize the real-time output voltage.

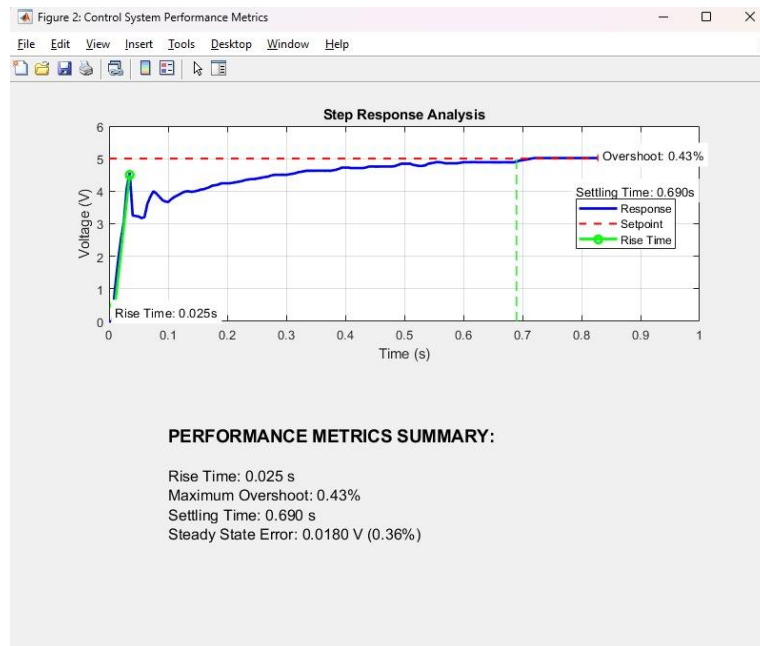


Figure 20. Step Response Analysis of the Voltage Control of Buck Converter using PID Controller

The step response depicted in Figure 20 shows that the classical PID controller successfully tracks the reference voltage with acceptable accuracy. However, a small overshoot and moderate settling time were observed. These indicate that while the controller is effective for basic regulation, its performance is limited under dynamic conditions due to its static gain structure.

4.2. Fuzzy PID Controller Performance

In the second phase, the fuzzy PID controller was activated. The controller continuously adjusted the PID gains based on real-time error and error rate using the fuzzy inference mechanism. Figure 21 shows the GUI interface for this configuration, similar in functionality to the one used with the classical PID.

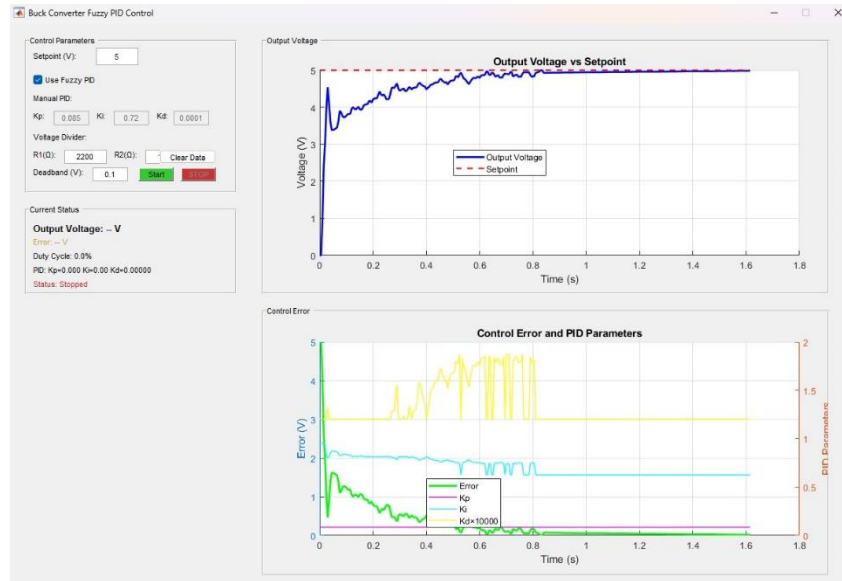


Figure 21. GUI of the Voltage Control of Buck Converter using Fuzzy PID Controller

The corresponding step response is provided in Figure 22. Compared to the classical PID, the fuzzy PID controller achieved faster rise time and eliminated overshoot entirely. The system transitioned more smoothly and demonstrated better stability, particularly in the presence of load changes or reference variation.

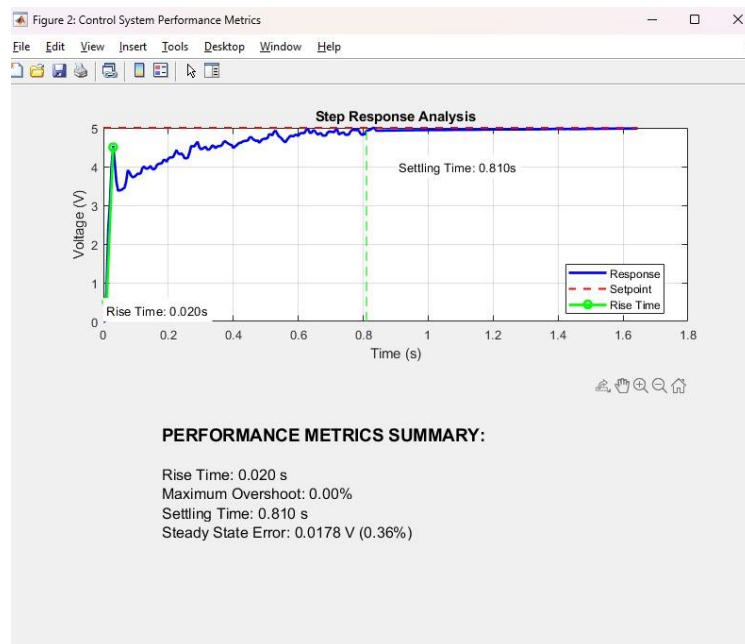


Figure 22. Step Response Analysis of the Voltage Control of Buck Converter using Fuzzy PID Controller

4.3. Comparative Analysis

When the step responses of the two controllers are examined side by side, it is clear that the fuzzy PID controller outperforms the classical PID in dynamic performance. The adaptive gain tuning provided by the fuzzy inference system enables faster, smoother, and more stable regulation. Although the settling time was slightly longer in some cases, the absence of

overshoot and reduced rise time make fuzzy PID a more robust and responsive solution for real-time voltage control.

	PID Controller	Fuzzy PID Controller
Rise Time	0.025 s	0.020 s
Maximum Overshoot	0.43 %	0.00 %
Settling Time	0.690s	0.810s
Steady State Error	0.0180V (0.36%)	0.0178V (0.36%)

5. Discussion

In this study, the voltage regulation performance of a Buck Converter was analysed using both a classical PID controller and a fuzzy logic-based PID controller implemented on the STM32F429ZI microcontroller. The controllers were first modelled and tuned in the MATLAB environment and then deployed to a real-time embedded system. Experimental step response tests were conducted to compare performance in terms of rise time, overshoot, settling time, and steady-state accuracy.

The classical PID controller provided acceptable results in terms of stability and steady-state error under nominal conditions. However, it exhibited limited adaptability when the system experienced dynamic changes, such as load disturbances or rapid reference transitions. This limitation stems from its fixed gain parameters, which cannot account for varying system behaviour.

The fuzzy PID controller, on the other hand, dynamically adjusted its gain values in real time based on the system error and its rate of change. This resulted in noticeably faster rise times, elimination of overshoot, and smoother transient response. Although the settling time was slightly longer in some cases, the overall control performance was significantly improved due to the adaptive and nonlinear nature of the fuzzy inference system.

These findings demonstrate that fuzzy logic-enhanced control offers a viable and effective alternative to classical control strategies in embedded power applications. For future work, the fuzzy controller can be further improved by incorporating learning algorithms such as genetic optimization or adaptive neuro-fuzzy tuning to refine the rule base and membership functions in real time.

6. Conclusion & Future Work

This study presented the design, implementation, and experimental evaluation of classical PID and fuzzy logic-based PID controllers for regulating the output voltage of a Buck Converter. The control systems were first modelled in MATLAB and then implemented on the STM32F429ZI microcontroller. Real-time tests were conducted under step reference input and sudden load disturbances to assess performance.

The classical PID controller demonstrated acceptable results in terms of steady-state accuracy but struggled with overshoot and slower recovery during dynamic transitions. In contrast, the fuzzy PID controller achieved a faster rise time, completely eliminated overshoot, and showed superior disturbance rejection. These improvements are attributed to its adaptive structure, where PID gains are continuously adjusted based on real-time error and error rate using a fuzzy inference mechanism.

The findings indicate that fuzzy logic can significantly enhance control system performance in nonlinear or time-varying environments, especially in embedded power applications where robustness is critical. However, the fuzzy controller requires more design effort and introduces additional computational complexity, which should be considered when selecting hardware platforms.

For future work, optimization techniques such as Genetic Algorithms (GA), Particle Swarm Optimization (PSO), or Adaptive Neuro-Fuzzy Inference Systems (ANFIS) could be integrated to automate the tuning of membership functions and rule bases. Additionally, applying the fuzzy PID approach to multi-phase converters or more complex power electronic topologies could extend its utility. Further analysis of computational load, memory usage, and execution time on resource-constrained microcontrollers would also help evaluate the scalability of the proposed method.

7. References

- [1] G. Mustafa and I. Jr, "Review on Performance Analysis of Three Control Techniques for Buck Converter feeding a Resistive Load," vol. 4, pp. 39–51, Jun. 2022.
- [2] Mustofa, J. Jumiyatun, and . M., "Controlling DC-DC Buck Converter Using Fuzzy-PID with DC motor load," *IOP Conf Ser Earth Environ Sci*, vol. 156, p. 012003, May 2018, doi: 10.1088/1755-1315/156/1/012003.
- [3] "MATLAB." Accessed: Jun. 16, 2025. [Online]. Available: <https://www.mathworks.com/products/matlab.html>
- [4] K. F. Hussein, I. Abdel-Qader, and M. K. Hussain, "Hybrid fuzzy PID controller for buck-boost converter in solar energy-battery systems," in *2015 IEEE International Conference on Electro/Information Technology (EIT)*, 2015, pp. 70–75. doi: 10.1109/EIT.2015.7293323.
- [5] H. Yang, L. Liu, Z. Yan, and X. Yang, "A fuzzy PID control strategy for buck converter system of double closed loop circuits," vol. 50, pp. 35–40 and 67, Apr. 2016, doi: 10.7652/xjtuxb201604006.
- [6] N. Wahid and N. Hassan, "Self-Tuning Fuzzy PID Controller Design for Aircraft Pitch Control," *Proceedings - 3rd International Conference on Intelligent Systems Modelling and Simulation, ISMS 2012*, Feb. 2012, doi: 10.1109/ISMS.2012.27.